

SoundSight

AI-Powered Sound Awareness for the Hearing Impaired

Combined Final Exam Project — AI Apps + IoT

Revision v2 — aligned with AI Apps subject requirements

This document outlines the full project plan including pitch, features, architecture, AI orchestration, timeline, and work division.

Tech Stack: ESP32 (C/C++) • Python 3.11 / FastAPI • React + TypeScript • Ollama + LangChain • MCP • ChromaDB (RAG) • Promptfoo • pytest / Vitest

Hardware: ESP32 WROOM • INMP441 I2S mic • WS2812B LEDs • Vibration motor

Table of Contents

- 1. Project Pitch
- 2. Problem Statement
- 3. Target Audience
- 4. Core Features
- 5. Technical Architecture
- 6. ML Pipeline
- 7. ESP32 Firmware Flow
- 8. API Endpoints
- 9. LangChain Orchestration
- 10. MCP Service Layer
- 11. RAG Design
- 12. Memory Design
- 13. Prompt Engineering & Techniques
- 14. Testing Strategy (Promptfoo + Unit Tests)
- 15. Project Timeline
- 16. Demo Scenario
- 17. Division of Work
- 18. Future Expansion
- 19. Key Selling Points
- 20. AI Apps Requirements Coverage Matrix

1. Project Pitch

There are approximately 430 million people worldwide living with disabling hearing loss. Every day, they miss critical sounds — a fire alarm, a doorbell, a baby crying, a kettle boiling over. Existing assistive solutions are either prohibitively expensive, limited in scope, or require proprietary ecosystems.

SoundSight is an affordable, intelligent sound awareness system that turns any room into a hearing-accessible environment. Using a low-cost microcontroller (~\$5 ESP32), on-device audio capture, machine learning classification, and AI-powered contextual intelligence, SoundSight detects, categorizes, and communicates environmental sounds through visual alerts (LEDs), haptic feedback (vibration), and a fully accessible web dashboard — all in real time.

What sets SoundSight apart is not just detection, but understanding. The system doesn't just say "sound detected" — it tells you "Your doorbell rang. This is the third time today, and it's consistent with your usual delivery window." It learns patterns, flags anomalies, and generates plain-language reports. The AI layer acts as an intelligent interpreter, not a chatbot.

For the exams: This project demonstrates competency across both AI Apps and IoT coursework. The IoT layer covers real-time sensor data capture, WiFi communication, multi-node architecture, and actuator control. The AI Apps layer covers audio feature engineering, ML classification, LLM orchestration via LangChain, a local Ollama-hosted model, an MCP service layer, RAG, memory, prompt engineering, and prompt regression testing with Promptfoo.

2. Problem Statement

Hearing-impaired individuals face daily risks and inconveniences because they cannot perceive environmental sounds. Current solutions have significant gaps:

- Dedicated alerting devices (e.g. flashing doorbell kits) are single-purpose and expensive when you need multiple types.
- Hearing aids and cochlear implants help with speech, but don't solve environmental awareness — especially at night when devices are removed.
- Smartphone sound recognition (e.g. Apple's Sound Recognition) works, but only when the phone is nearby, charged, and not in a bag.
- No existing solution combines detection, contextual intelligence, multi-room coverage, and affordability in one system.

SoundSight fills this gap with a modular, scalable, and intelligent approach.

3. Target Audience

- **Primary:** People with moderate to profound hearing loss living independently or with family.
- **Secondary:** Elderly individuals with age-related hearing decline.
- **Tertiary:** Parents wanting an extra layer of monitoring (e.g. baby crying detection in another room).
- **Institutional:** Care homes, schools for the deaf, hospitals — anywhere sound awareness improves safety.

4. Core Features

4.1 Real-Time Sound Detection and Classification

The system continuously listens through a digital microphone (INMP441) connected to the ESP32. Audio is captured in short windows (1–2 seconds) and sent to the backend for classification.

Critical / Safety Sounds:

- Fire alarm / smoke detector
- Glass breaking
- Doorbell / door knocking
- Car horn
- Baby crying

Everyday Awareness Sounds:

- Kettle / microwave / oven timer beeping
- Phone ringing
- Water running (tap left on)
- Dog barking
- Clapping / attention-getting sounds

Future Expansion:

- Washing machine / dryer finished
- Alarm clock in another room
- Thunderstorm / severe weather sounds

For the exam scope, we target 6–8 well-detected classes. The architecture supports adding more.

4.2 Multi-Modal Alerting

Hearing-impaired users can't rely on audio alerts, and they might not always be looking at a screen. SoundSight uses multiple alert channels:

- **Visual (LED strip):** Color-coded per sound type. Red = fire alarm, blue = doorbell, yellow = timer, green = general. LEDs persist for several seconds.
- **Haptic (vibration motor):** Buzzes for critical events. Different patterns per severity (short pulse = informational, continuous = urgent).
- **Push notification:** For critical events, a notification is sent even if the user isn't near the device.
- **OLED display (optional):** Shows last detected event, timestamp, and confidence.

4.3 Web Dashboard (React Frontend)

Live View:

- Real-time feed of detected sounds: event type, timestamp, confidence score, source room/device.
- Current status per room: "Kitchen: quiet" / "Front door: knocking detected 2 min ago."
- Color-coded event cards matching the LED color scheme.

Configuration:

- Enable/disable sound classes based on personal relevance.

- Set priority levels: critical vs. informational per sound type.
- Quiet hours: "Don't alert for doorbell between 23:00–07:00, unless safety event."
- Device/room management: name devices, assign to rooms.

History and Analytics:

- Timeline view of all events, filterable by type, room, date.
- Trend charts: sound frequency by type, time of day, room.
- Weekly comparison: "This week vs. last week" overview.

Accessibility (non-negotiable):

- High contrast mode and large text options.
- Fully keyboard-navigable.
- No audio-dependent UI feedback anywhere.
- Clean, simple layout — assume the user may not be technical.

4.4 AI-Powered Intelligence Layer (LLM — Non-Chatbot)

The LLM is used as an **analytical tool**, not a conversational agent. It runs locally via **Ollama**, is orchestrated by **LangChain**, and pulls context through an **MCP service layer** and **RAG retrievers**. It has five distinct roles:

a) Event Interpretation — Turns raw ML classifications into context-rich descriptions.

Example: raw data says `water_running` for 47 minutes at 03:22 in the bathroom. LLM output: "Water has been running in the bathroom for 47 minutes starting at 03:22 AM. This is unusual — your typical water usage at night is under 5 minutes. This may indicate a forgotten tap."

b) Report Generation — Daily and weekly summaries. The agent calls MCP tools to query the event DB and retrieves similar past events via RAG before composing the narrative.

c) Natural Language Rule Parsing — Users describe alert rules in plain text. The chain uses a Pydantic output parser to emit a strict JSON rule: trigger, time window, priority, alert type. **Example:** "Wake me up if the fire alarm goes off at night" → structured rule.

d) Anomaly Narration — When the ML/statistics layer flags anomalies, the agent pulls the user's baseline (memory) + similar historical events (RAG) and explains the deviation in human terms.

e) Onboarding Profiler — During first setup, the user describes their home. The chain emits a structured sound profile config (enabled classes, priorities, default quiet hours).

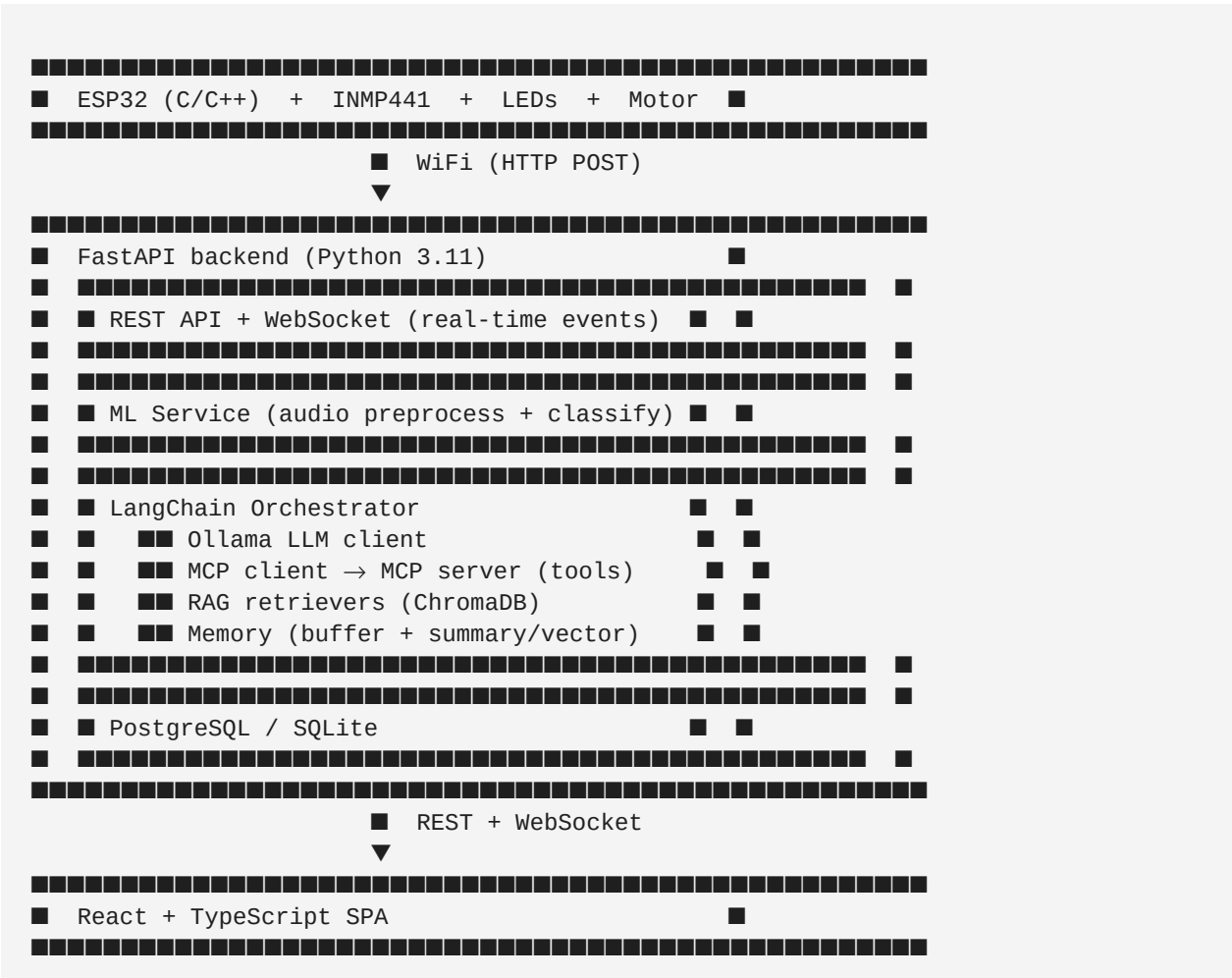
4.5 Multi-Node Support

Each ESP32 is a "node" assigned to a room. The architecture supports multiple nodes reporting to the same backend. Even if we build one physical device for the exam, the software handles multiple devices: self-registration on first boot, room-by-room dashboard view, and room-specific alert rules.

5. Technical Architecture

5.1 System Overview

The system follows a layered architecture with clear separation of concerns:



5.2 Tech Stack

Layer	Technology
Microcontroller	ESP32 (WROOM or S3) — firmware in C/C++
Microphone	INMP441 I2S digital mic
Actuators	WS2812B LED strip, vibration motor, SSD1306 OLED (optional)
Backend API	Python 3.11 + FastAPI (REST + WebSocket)
LLM runtime	Ollama (local) — standard model or custom via Modelfile
LLM orchestration	LangChain (LCEL chains + agents + tools)
Context / tools	MCP server(s) exposed by backend
RAG	ChromaDB + sentence-transformers embeddings
Memory	LangChain memory (buffer + summary + vector-backed)
ML training	Python (librosa, scikit-learn / PyTorch)

ML inference	Same Python stack served in-process by FastAPI
Database	PostgreSQL (production) or SQLite (exam demo)
Frontend	React + TypeScript (Vite)
Real-time comms	FastAPI WebSocket
Device-to-server	HTTP POST (raw audio bytes)
Prompt testing	Promptfoo (golden set + rubric tests)
Unit testing	pytest (Python), Vitest + RTL (React)

5.3 Why the stack changed from v1

The v1 plan used C#/.NET 8 for the backend. v2 swaps this for Python + FastAPI to align with the AI Apps subject requirements. This simplifies the system considerably:

- ML training and inference now share one language and one preprocessing pipeline (librosa) — no more Python→ONNX→C# round-trip.
- LangChain, Ollama clients, LangChain-MCP adapters, ChromaDB, and sentence-transformers are Python-native.
- SignalR (C#-centric) is replaced by FastAPI's native WebSocket support. If we need fan-out/rooms, we can drop in python-socketio with minimal effort.
- Audio preprocessing (mel-spectrograms, MFCC) stays exactly as-is from training code.

6. ML Pipeline

6.1 Training (Python, Offline)

- Collect/source audio samples — ESC-50 dataset, UrbanSound8K, and/or custom recordings.
- Preprocess: resample to 16 kHz mono, split into 1–2 second clips.
- Feature extraction: mel spectrograms or MFCCs via librosa.
- Train classifier: CNN on spectrograms, or Random Forest / SVM on MFCC features.
- Evaluate: confusion matrix, precision/recall per class, overall accuracy.
- Save as a native Python artifact (e.g. a PyTorch state_dict or a joblib-pickled sklearn pipeline). ONNX export is optional and no longer required.

6.2 Inference (Python, Runtime)

- FastAPI endpoint receives raw PCM audio bytes from the ESP32.
- Audio preprocessing (same code as training — resample, window, feature extract).
- Run the loaded classifier in-process.
- Return classification result with confidence score.
- If confidence is below threshold → label as "unknown" rather than guessing.
- Publish the event over WebSocket to the dashboard and trigger the LLM pipeline (LangChain) for interpretation.

6.3 Training Data Sources

- ESC-50: 2,000 environmental audio clips in 50 classes (freely available).
- UrbanSound8K: 8,732 labeled urban sound clips.
- AudioSet (Google): massive dataset, cherry-pick relevant classes.
- Custom recordings: our own doorbell, smoke detector, etc. for domain-specific accuracy.

7. ESP32 Firmware Flow

Firmware is written in C/C++ using the Arduino framework on PlatformIO. The main loop:

```
loop():
  1. Capture 1–2 seconds of audio via I2S from INMP441
  2. Buffer the PCM data
  3. POST raw audio bytes to backend endpoint
  4. Receive response: { event_type, severity, led_color, vibration_pattern }
  5. Drive LEDs: set color, hold for N seconds
  6. Drive vibration motor: pulse pattern based on severity
  7. Update OLED display (if present): event type + timestamp
  8. Wait for next capture window (configurable, e.g. 500 ms)
```

WiFi credentials and backend URL are configured via a simple config portal on first boot (ESP32 AP mode). Firmware is unchanged from v1 — only the backend endpoint it talks to has moved from C# to Python.

8. API Endpoints

All endpoints are FastAPI routes. Real-time event push uses a WebSocket channel instead of SignalR. Paths are unchanged from v1 where possible.

Method	Endpoint	Description
POST	/api/audio/classify	ESP32 sends audio, gets classification back
GET	/api/events	Paginated event history
WS	/ws/events	WebSocket channel for real-time event stream
GET	/api/reports/daily	LLM-generated daily digest (LangChain + Ollama)
GET	/api/reports/weekly	LLM-generated weekly digest
POST	/api/rules	Create alert rule (structured or NL → parsed by LangChain)
GET	/api/rules	List active rules
PUT	/api/settings	Update user preferences
POST	/api/devices/register	Device self-registration on first boot
GET	/api/devices	List registered devices
GET	/api/dashboard/summary	Aggregated stats for dashboard home
POST	/api/onboarding/profile	Run onboarding profiler chain, return sound profile
POST	/api/events/{id}/explain	On-demand LLM explanation for a specific event

9. LangChain Orchestration

LangChain is the backbone of the AI layer. Each of the five LLM roles from §4.4 is a discrete LangChain artifact (chain or agent). All of them target Ollama via `langchain_ollama.ChatOllama`.

9.1 Chain/Agent Map

LLM Role	LangChain Construct	Key Components
Event interpretation	LCEL chain	PromptTemplate → ChatOllama → Pydantic output parser
Report generation	Agent	ChatOllama + MCP tools (<code>query_events</code> , <code>get_device_status</code>) + RAG retriever +
NL rule parsing	LCEL chain	Few-shot PromptTemplate → ChatOllama → Pydantic(Rule) parser
Anomaly narration	Agent	ChatOllama + long-term memory (baselines) + RAG (similar past events)
Onboarding profiler	LCEL chain	PromptTemplate → ChatOllama → Pydantic(SoundProfile) parser

9.2 Example: Rule Parsing Chain (sketch)

```
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field
from typing import Literal

class Rule(BaseModel):
    trigger: str = Field(description="sound class name")
    time_start: str | None
    time_end: str | None
    priority: Literal["info", "warn", "critical"]
    alert_type: list[Literal["led", "vibration", "push"]]

parser = PydanticOutputParser(pydantic_object=Rule)
llm = ChatOllama(model="llama3.1:8b", temperature=0)
prompt = ChatPromptTemplate.from_messages([
    ("system", SYSTEM_PROMPT + parser.get_format_instructions()),
    ("user", "{nl_rule}")
])
chain = prompt | llm | parser
```

9.3 Which LLM model

Default: a stock Ollama model — `llama3.1:8b-instruct` or `qwen2.5:7b-instruct` — with role-specific system prompts layered at runtime. This satisfies the requirement without bespoke model work.

Optional upgrade: a custom Ollama model built via `Modelfile`, baking in the shared `SoundSight` base persona. If we take this route, the `Modelfile` is handed in as part of the deliverable. Example shape:

```
# Modelfile — soundsight-base
FROM llama3.1:8b-instruct
PARAMETER temperature 0.2
PARAMETER num_ctx 8192
SYSTEM """
You are SoundSight's analysis layer. You interpret environmental sound events
for hearing-impaired users. You are factual, calm, and concise. You never invent
data; if you lack information, you say so. When asked for structured output,
you emit valid JSON only, with no surrounding prose.
"""
```

Final decision (stock vs custom) is pushed to Week 3, once we have empirical prompt data from Promptfoo runs against both options.

10. MCP Service Layer

The Model Context Protocol (MCP) gives the LangChain agents a clean, standardized way to pull context and trigger actions against the backend. The backend hosts an MCP server; LangChain acts as the MCP client (via langchain-mcp-adapters).

10.1 Why MCP here

- Separates **LLM reasoning** from **data access**. The LLM never talks directly to PostgreSQL.
- Tools are versioned, discoverable, and typed — easy to test in isolation.
- If the same tools are later exposed to Claude Desktop or another MCP client, no rewrite needed.

10.2 Tools exposed by the SoundSight MCP server

Tool	Signature (simplified)	Used by
query_events	(filter, time_range) → list[Event]	Report generation, anomaly narration
get_device_status	(device_id) → DeviceStatus	Event interpretation, report generation
get_user_profile	() → UserProfile	Event interpretation, rule parsing
get_active_rules	() → list[Rule]	Rule parsing (conflict check), report generation
get_sound_class_info	(class_name) → ClassInfo	Event interpretation (borderline cases)
get_user_baseline	(class_name, window) → Baseline	Anomaly narration

10.3 Architecture note

The MCP server runs in-process with FastAPI during the exam (simplest deployment) but is implemented against the standard MCP spec so it can be split into its own process later without code changes to the agents.

11. RAG Design

Three retrieval-augmented generation use cases are layered into the AI pipeline. All three use ChromaDB (embedded) for vector storage and sentence-transformers (all-MiniLM-L6-v2) for embeddings.

11.1 Event History RAG

- **Corpus:** natural-language summaries of past events (auto-written after each classification).
- **Embedding key:** class + time-of-day bucket + duration + free-text context.
- **Used by:** report generation, anomaly narration.
- **Why it helps:** grounds narratives in real history — "last Tuesday's similar event was cooking-related, lasted 8 seconds."

11.2 Home Knowledge RAG

- **Corpus:** the onboarding profile + any user-added notes (pets, layout, routines).
- **Used by:** event interpretation, anomaly narration.
- **Why it helps:** gives the LLM stable facts about the user ("lives alone, has a dog, wired doorbell") without stuffing them into every prompt.

11.3 Sound-Class Knowledge Base RAG

- **Corpus:** curated descriptions per sound class — typical scenarios, safety implications, common false positives.
- **Used by:** event interpretation (medium-confidence cases).
- **Why it helps:** when the classifier is uncertain, the LLM has reference material to reason with ("a 0.6-confidence kettle beep at 7 AM in the kitchen is very likely a real kettle, not a smoke detector").

11.4 Indexing lifecycle

- Every classified event is embedded and written to the event-history collection.
- Onboarding profile is re-indexed whenever the user updates it.
- Sound-class KB is a static, versioned asset shipped with the repo.

12. Memory Design

Memory is a natural fit for SoundSight — patterns and baselines are already core to the product value proposition. We split memory into two layers, both via LangChain primitives.

12.1 Short-term memory (session buffer)

- **Type:** ConversationBufferMemory or ConversationBufferWindowMemory.
- **Scope:** a single user session — e.g. interactive rule creation where the user iterates on phrasing.
- **Lifetime:** cleared when the session ends.

12.2 Long-term memory (user baselines)

- **Type:** vector-backed memory (Chroma) + periodic summary memory.
- **Scope:** per-user, persistent.
- **Content:** rolling summaries of sound activity — "typical doorbell window is 14:00–16:00", "usual night-time water usage is <5 minutes".
- **Lifetime:** permanent; updated on a schedule (nightly job) that re-summarizes the last N days of events.
- **Consumers:** anomaly narration (compares current event against baseline), report generation (uses baselines as reference points).

12.3 Why not memory everywhere

Rule parsing, event interpretation, and onboarding profiler are **stateless** on purpose. Each call should be reproducible given only its inputs. Mixing memory into stateless chains would make Promptfoo testing unreliable and the rule-parsing JSON output non-deterministic.

13. Prompt Engineering & Techniques

13.1 Shared base system prompt

All five roles share a single base system prompt defining persona, tone, and hard rules (factual, calm, concise, JSON-only when asked, no hallucinations, no invented data). Role-specific instructions are appended per chain.

13.2 Technique per role

Role	Techniques applied
Event interpretation	System prompt, few-shot (before/after enrichment examples), MCP tool use
Report generation	System prompt, CoT (outline → narrative), tool use, RAG
NL rule parsing	System prompt, few-shot (3–5 example rules), Pydantic structured output, temperature 0
Anomaly narration	System prompt, CoT (reason about why unusual before narrating), RAG, memory
Onboarding profiler	System prompt, few-shot, Pydantic structured output

13.3 Output-format discipline

- Any chain emitting structured data uses a Pydantic model + PydanticOutputParser. No freeform JSON parsing.
- Temperature 0 for rule parsing and onboarding profiler; 0.2–0.4 for narrative roles.
- Every role's prompt includes an explicit refusal clause: "If you don't have enough information, return the field as null rather than inventing a value."

14. Testing Strategy

14.1 Promptfoo (prompt regression testing)

A promptfoo/ directory at the repo root contains a YAML test suite per LLM role. Tests run locally against the same Ollama instance used in development.

- **Rule parser:** golden input/output pairs (strict JSON match). Covers affirmative phrasing, negations, ambiguous time phrases, multi-trigger rules.
- **Event interpretation:** rubric-based tests — does the output mention duration? user context? avoid inventing facts?
- **Anomaly narration:** tests that the output references the baseline and does not assert false urgency.
- **Onboarding profiler:** golden pairs — home descriptions → expected enabled classes.
- **Regression gate:** Promptfoo runs in CI (or at minimum, a pre-demo script) and blocks merges on rule-parser regressions.

14.2 Unit tests

Component	Framework	What we test
Audio preprocessing	pytest	Resampling, windowing, MFCC/mel shape, silence handling
Classifier wrapper	pytest	Input shape validation, confidence threshold behavior, 'unknown' fallback
FastAPI endpoints	pytest + TestClient	Status codes, schemas, auth (if any), WebSocket round-trip
LangChain chains	pytest + mocked LLM	Prompt assembly, parser behavior, error paths
MCP tool handlers	pytest	Tool I/O contracts, DB query correctness
RAG retrievers	pytest	Top-k relevance on fixed corpus, embedding shape
React components	Vitest + RTL	Dashboard rendering, filter logic, accessibility props
ESP32 firmware	on-device manual + PlatformIO	Only on target, where feasible

14.3 Integration testing

- End-to-end smoke test: pre-recorded audio file → HTTP POST → classification → WebSocket event on dashboard.
- Chaos-style tests for edge cases: low confidence, network drops, unknown sounds.

15. Project Timeline (3–4 Weeks)

Week 1 — Foundation

- Set up FastAPI project (Python 3.11, uvicorn, pydantic v2, pytest).
- Set up React frontend with Vite, routing, and placeholder pages.
- ESP32 firmware: I2S mic capture + WiFi + HTTP POST working.
- Source and organize training data (ESC-50 + custom recordings).
- Database schema design + Alembic migration.
- Stand up local Ollama + pull base model; verify langchain_ollama connectivity.

Week 2 — ML Pipeline + Core Features

- Train classification model in Python, evaluate, save artifact.
- Integrate classifier into FastAPI inference endpoint.
- End-to-end flow: ESP32 → backend → classify → response → LEDs.
- Build dashboard live event feed (WebSocket).
- Event history page with filtering.
- Seed sound-class KB (static corpus) and stand up ChromaDB.

Week 3 — LLM Layer + RAG + MCP + Polish

- Build LangChain chains for event interpretation, rule parsing, onboarding profiler.
- Build LangChain agents for report generation and anomaly narration.
- Implement MCP server with the six core tools (§10.2).
- Wire up RAG retrievers (event history, home knowledge, sound-class KB).
- Implement memory layer (short-term session + long-term baselines).
- Configuration UI: sound toggles, priority levels, quiet hours.
- Device management page.
- Write Promptfoo test suite and iterate on prompts until golden sets pass.

Week 4 — Integration, Testing, Demo Prep

- Complete unit test coverage for core components (§14.2).
- Full integration testing: all layers working together.
- Edge case handling: low confidence, network drops, unknown sounds.
- Decide standard vs custom Ollama model based on Promptfoo data; if custom, finalize Modelfile.
- Demo scenario rehearsal with pre-recorded audio backups.
- Presentation slides and talking points for both exams.
- Documentation and code cleanup.

16. Demo Scenario (Exam Presentation)

Live demo flow (approximately 5–7 minutes):

1. Show the physical device — ESP32 with mic, LEDs, vibration motor.
2. Play a doorbell sound → LEDs turn blue, dashboard shows event in real time.

- 3. Play a fire alarm sound → LEDs turn red, vibration activates, push notification appears.
- 4. Play a timer beep → LEDs turn yellow, dashboard logs it.
- 5. Open dashboard: show live feed, event history, trend chart.
- 6. Show LLM-generated daily report — highlight a line that clearly used RAG ("similar to last Tuesday's event") and a line that clearly used memory/baselines.
- 7. Type a natural language rule: "Vibrate urgently if glass breaks after 22:00" → show the parsed JSON rule.
- 8. Show the Promptfoo run output for the rule parser — proof of prompt quality.
- 9. Show accessibility features: high contrast, large text, no audio dependencies.

Backup plan: If live audio detection is flaky during the demo, have pre-recorded clips ready on a phone speaker, or simulate events via a test endpoint that injects fake classifications.

17. Division of Work (Two Partners)

Partner A (Stefan)	Partner B (Samuel)
ESP32 firmware (C/C++: I2S, LEDs, vibration, WiFi)	FastAPI structure + REST + WebSocket
ML model training + inference wrapper	LangChain chains/agents + Ollama integration
Audio preprocessing (Python, shared with training)	MCP server + RAG pipeline (ChromaDB)
Hardware assembly + demo prep	React frontend (dashboard, config, history)
Promptfoo test suite	Memory layer + unit test infrastructure

Note: the two partners must be interchangeable on the specific column assignment — this is a suggested split, not a hard one. Both partners need to understand the full system for exam defense, especially the cross-cutting pieces (MCP, RAG, memory).

18. Future Expansion Ideas

Beyond exam scope but strengthen the presentation as "future work":

- **Edge ML:** Run lightweight TFLite model directly on ESP32-S3 for offline critical sound detection.
- **Custom sound training:** Users record their own doorbell, their own dog for personalized models.
- **Wearable form factor:** ESP32 in a wristband with vibration — always-on-body alerts.
- **Multi-language support:** LLM reports in the user's preferred language.
- **Caregiver mode:** A second dashboard for a family member or caregiver.
- **Smart home integration:** Trigger Philips Hue lights to flash on critical sounds.
- **Mobile app:** Native iOS/Android companion with push notifications.
- **Sound direction / localization:** Phase 1 — dual-mic GCC-PHAT for L/R azimuth per node; Phase 2 — multi-node RMS energy comparison for room-level localization (clock sync via WiFi NTP is too loose for cross-node TDOA).

19. Key Selling Points for Presentation

- **Real human impact:** Solves a genuine daily problem for 430 million people.

- **Affordable:** Total hardware cost under \$15 per node.
- **Intelligent, not just reactive:** LLM provides context, patterns, and explanations.
- **Modular and scalable:** Add rooms by adding \$5 nodes.
- **Accessible by design:** No audio-dependent UI anywhere.
- **Strong technical depth:** IoT protocols, real-time systems, ML classification, LangChain + Ollama orchestration, MCP tooling, RAG, memory, prompt engineering, full-stack web.
- **Testable AI:** Promptfoo regression suite proves the LLM layer is not a black box.
- **Privacy-friendly:** LLM runs locally via Ollama — no user audio or event data leaves the machine.

"There are approximately 430 million people worldwide with disabling hearing loss. Most smart home sound alerts require expensive proprietary hardware. Our system uses a \$5 microcontroller, open-source ML, and a locally-hosted LLM with RAG and memory to make any room sound-aware — affordably, accessibly, and privately."

20. AI Apps Requirements Coverage Matrix

This matrix maps every line of the AI Apps subject requirements to the section(s) of this plan that cover it. Intended as a quick cross-check for the examiner.

#	Requirement	Covered in
1	React frontend	§4.3, §5.2
2	Python backend as REST API using FastAPI	§5, §6.2, §8
3	One or more LLMs hosted in Ollama (custom → hand in Modelfile)	§5.2, §9.3
4	LLM controlled via system prompting and proper prompt techniques	§4.4, §13
5	LangChain for orchestrating models, prompts, agents, tools	§9
6	API between frontend and backend	§8
7	MCP between LangChain orchestration and services	§10
8	Prompt testing with Promptfoo	§14.1
9	Unit tests for all core components	§14.2
10	Memory (if it makes sense)	§12
11	RAG	§11
—	IoT: ESP32 + INMP441 controlled with C/C++	§5.2, §7